



SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL

Alice Andrade

Erick Santos

Nicolas Oliveira Lima

Vinicius Augusto

Nova Lima

2026

1. Introdução e Contextualização do Projeto

No cenário industrial contemporâneo, a rastreabilidade e a transparência ambiental tornaram-se pilares indispensáveis na cadeia de suprimentos da indústria automotiva pesada. A complexidade logística que envolve a manufatura de veículos exige um ecossistema tecnológico integrado de alta performance, capaz de catalogar insumos e monitorar a produção em tempo real. Ademais, as diretrizes de governança ambiental, social e corporativa (ESG) impõem requisitos rígidos de conformidade e auditoria sobre a emissão de gases de efeito estufa.

Este trabalho documenta o desenvolvimento técnico de uma solução abrangente voltada ao gerenciamento de suprimentos e ao cálculo automatizado da pegada de carbono para veículos industriais da marca Iveco. O sistema foi concebido como uma evolução de um protótipo inicial validado no Desafio SAGA SENAI, sendo agora inteiramente reestruturado e refatorado com base nas melhores práticas da Engenharia de Software assimiladas no curso técnico de Desenvolvimento de Sistemas.

A arquitetura original do projeto baseava-se em um padrão monolítico monolocalizado com persistência relacional básica. Visando aproximar a solução das demandas de um cenário de produção em escala industrial, a nova infraestrutura foi distribuída em microsserviços e aplicações complementares interligadas, separando as responsabilidades de captura, intermediação lógica e exibição visual rica dos dados coletados na fábrica.

2. Arquitetura do Sistema e Stack Tecnológica

A solução foi dividida estrategicamente em três subprojetos independentes que formam o ecossistema e se comunicam de maneira segura, sob o suporte de um banco de dados hospedado em nuvem corporativa. A tabela abaixo resume as camadas do sistema e as respectivas ferramentas tecnológicas associadas:

Tabela 1: Matriz de Componentes e Stack Tecnológica do Ecossistema

Componente	Tecnologia	Padrão / Framework	Função no Ecossistema
ApiIveco (Back-End)	.NET 8 / ASP.NET	REST API	Centraliza regras de negócio e isola o acesso seguro ao banco de dados.
Wpflveco (Front-End)	C# / WPF	MVVM	Interface visual reativa para monitoramento e controle estatístico.
SimuladorIveco (Utilitário)	C# (Console)	Task Parallel Library	Simula dados de telemetria e sensores industriais em tempo real.
Firebase Firestore (Banco)	Google Cloud	NoSQL	Persistência escalável e assíncrona orientada a documentos.

Para garantir a homogeneidade visual, padronização de bibliotecas e o reaproveitamento de rotinas e tratamento assíncrono avançado no ecossistema .NET, as seguintes ferramentas específicas e metatags estruturais foram consolidadas no ambiente de engenharia:
Tecnologias Utilizadas: C# 12 | .NET 8 LTS | ASP.NET Core Web API | WPF Desktop | MVVM Toolkit | Firebase Firestore SDK | LiveCharts2 | SkiaSharp Engine | Swagger OpenAPI | RESTful Client JSON | NoSQL Architecture | Google Cloud IAM | Async / Await Pattern | Parallel Task Pool

2.1. Topologia de Pastas e Distribuição Arquitetural

O fluxo estrutural de acoplamento do sistema é hierárquico e segregado, impedindo que a interface gráfica execute chamadas diretas ao repositório de persistência. A disposição das camadas reflete a implementação rigorosa da engenharia de software distribuída:

Estrutura Gráfica do Fluxo da Arquitetura do Software

Etapa	Camada do Sistema	Componentes e Arquivos	Método de Comunicação
1	View (WpfIveco)	MainWindow.xaml (Interface Gráfica)	↓ <i>Data Binding</i>
2	ViewModel	MainViewModel.cs, RelayCommand.cs, DadosService.cs	↓ <i>Requisições HTTP</i>
3	API (ApiIveco)	DadosController.cs, FireBaseData.cs, Models (Entidades)	↓ <i>gRPC Seguro</i>
4	Nuvem (Persistência)	Firebase Firestore (NoSQL), IAM (firebase-key.json)	—

3. Migração de Infraestrutura e Gerenciamento de Crises

Durante o ciclo de desenvolvimento, a equipe de engenharia deparou-se com um incidente crítico envolvendo o sistema de controle de versão distribuído (Git) e a integridade da árvore de rastreamento do banco de dados local. Ao tentar consolidar uma refatoração estrutural complexa para a separação em camadas MVVM, conflitos de mesclagem (merge conflicts) severos corromperam os metadados locais de migração e dessincronizaram o histórico de commits do repositório principal.

Como medida imediata de mitigação do risco de interrupção e visando manter a rastreabilidade histórica, o repositório original afetado foi congelado para auditoria técnica e mantido publicamente disponível no endereço eletrônico abaixo:

URL do Repositório Histórico Auditado :

<https://github.com/NicolasOlim/Implementacao-do-MVVM-no-Projeto-Iveco.git>

Diretriz de Contingência e Arquitetura de Resiliência: A quebra da infraestrutura relacional local baseada em SQLite, embora representasse um obstáculo imediato, foi convertida em um catalisador de inovação tecnológica. A equipe optou por descontinuar

permanentemente o armazenamento acoplado local e realizar a migração integral da persistência para uma arquitetura orientada a documentos distribuídos em nuvem.

3.1. Reestruturação do Modelo de Dados para NoSQL

A transição do modelo relacional clássico de terceira forma normal (3NF) para a persistência em documentos não relacionais (NoSQL) exigiu uma quebra de paradigma na modelagem das entidades. Ao invés de tabelas rígidas ligadas por chaves estrangeiras restritivas, utilizou-se o modelo de coleções elásticas do Firebase Firestore, organizadas sob documentos JSON estruturados capazes de se expandirem de forma dinâmica.

Para manter a integridade analítica e possibilitar a consolidação de relatórios estatísticos sem o recurso nativo de comandos JOIN, foram mantidos identificadores textuais únicos (UUID) como elos de amarração lógica entre os documentos das coleções fornecedores, veículos e lotes.

4. Metodologia de Desenvolvimento e Protocolos de Rede

Para garantir a estabilidade do ecossistema distribuído, a camada de Back-End foi projetada sob o paradigma da programação paralela e assíncrona baseada nas palavras-chave nativas `async` e `await` do C#. A manipulação eficiente dos fluxos de execução impede que o servidor sofra de inanição de threads (thread starvation) quando submetido a altas taxas de concorrência.

4.1. Mecânica de Funcionamento do Thread Pool do Servidor

A engenharia de requisições concorrentes entre o gerador automático de carga (Simulador) e a interface administrativa (Dashboard Desktop) é processada conforme o fluxo otimizado abaixo:

1. O **SimuladorIveco** dispara de forma ininterrupta requisições assíncronas do tipo HTTP POST encapsulando a leitura de telemetria dos sensores industriais. Concorrentemente, o painel **Wpflveco** efetua requisições HTTP GET em busca de dados estatísticos agregados.
2. O servidor de aplicação ASP.NET Core intercepta as requisições e aloca uma thread leve originária do seu *Thread Pool* interno para iniciar o processamento de validação da requisição.
3. No momento em que a API dispara a persistência de rede para o Google Cloud (Firebase Firestore), o padrão assíncrono libera imediatamente a thread alocada de volta ao *Thread Pool* do sistema operacional. A thread não fica ociosa bloqueando recursos de hardware enquanto aguarda a resposta do link de rede de longa distância (WAN).
4. Durante o processamento da operação de entrada e saída (I/O) assíncrona na nuvem, o servidor permanece totalmente disponível para processar novos registros provenientes de outros sensores da planta fabril.
5. Ao receber o sinal de conclusão (callback gRPC) enviado pela infraestrutura do Google Firebase, o ecossistema .NET escala qualquer thread que esteja livre no pool no exato milissegundo para encapsular os metadados em formato JSON e finalizar a comunicação com o cliente original com o status de sucesso HTTP 200 OK.

5. Análise Detalhada das Camadas do Projeto

5.1. Camada de Serviços Intermediários - API (ApiIveco)

A ApiIveco atua como a inteligência central e o núcleo de governança do ecossistema. Construída com a especificação técnica do ASP.NET Core Web API em ambiente .NET 8, ela baseia-se nas restrições de projeto do estilo arquitetural REST (Representational State Transfer). A comunicação externa opera sobre os protocolos seguros HTTP/HTTPS utilizando objetos de transferência de dados (DTO) serializados estritamente em formato JSON.

5.2. Camada da Interface Visual - WPF (Wpflveco)

A aplicação Wpflveco representa a interface homem-máquina (IHM) de controle administrativo. O desenvolvimento em Windows Presentation Foundation (WPF) adota o padrão de arquitetura MVVM (Model-View-ViewModel), garantindo isolamento total entre os elementos gráficos (Views declaradas em sintaxe XAML) e o código de manipulação de dados em C# (ViewModels).

A sincronização reativa do painel é governada por mecanismos de Data Binding bidirecionais suportados pela interface de infraestrutura `INotifyPropertyChanged`. A interceptação de cliques e ações do usuário utiliza o padrão Command (via classes derivadas de `ICommand`, como o `RelayCommand`), erradicando qualquer lógica de persistência ou chamadas pesadas de rede de dentro dos arquivos de retaguarda das telas (code-behind), o que preserva a fluidez da linha de renderização da interface e elimina congelamentos visuais. Os dados coletados são plotados graficamente através da biblioteca estatística de alta performance LiveCharts2, acelerada via hardware gráfico por meio do motor de renderização vetorial SkiaSharp.

5.3. Camada de Simulação de Sensores (SimuladorIveco)

O módulo SimuladorIveco consiste em um motor utilitário em modo Console responsável por simular o comportamento operacional de dispositivos de telemetria automotiva e sensores IoT da linha de montagem (tais como antenas de identificação por radiofrequência - RFID, balanças eletrônicas automatizadas e scanners industriais de rastreamento de chassis). O utilitário gera pacotes sintéticos contendo números de chassi válidos, peso líquido das cargas transportadas e a tipologia dos insumos recebidos dos fornecedores externos.

Para garantir a resiliência operacional em cenários reais de falha em redes industriais, o simulador incorpora um mecanismo de contingência local de armazenamento (banco de dados embarcado leve `dados_gerados.db` em formato SQLite). Caso ocorra uma oscilação na infraestrutura de rede ou indisponibilidade temporária nos endpoints da API central, os dados gerados pelos sensores são enfileirados localmente no banco embarcado. Assim que a conectividade com os serviços REST da API é reestabelecida, o simulador processa o envio assíncrono em lotes (batch processing), garantindo risco zero de perda de telemetria e fornecendo insumos fidedignos para os testes de carga, concorrência e auditoria de escalabilidade da solução em nuvem.

6. Considerações Finais

O desenvolvimento do Sistema de Rastreamento Inteligente consolidou de forma prática os conceitos avançados de programação orientada a objetos, arquitetura distribuída,

programação assíncrona concorrente e modelagem de bancos de dados orientados a documentos em nuvem.

A transição emergencial provocada pela instabilidade no controle de versão original permitiu demonstrar a flexibilidade da equipe de engenharia e a resiliência do modelo em camadas adotado. O isolamento das responsabilidades por meio de uma API RESTful garantiu que a substituição da infraestrutura do banco de dados SQLite local pelo Google Firebase Firestore em nuvem fosse executada com impacto mínimo nas regras de apresentação gráfica do cliente WPF. A solução apresenta-se aderente aos requisitos de rastreabilidade modernos e pronta para atuar como ferramenta estratégica de monitoramento logístico e conformidade de pegada ecológica da frota Iveco.